

☰ Hide menu

Course Introduction

What is a Linear Program?

Tutorial: Solving Linear Programs Using PuLP Library

✔ Lab: Lab: Formulate and Solve LPs using PULP 1h

✔ Quiz: Solving LPs using PULP Submitted

Network Flow Problems and Linear Programming

Geometry of Linear Programs

Algorithms for Linear Programs: A brief overview

Problem Set # 1

Solving LPs using PULP

Review Learning Objectives

✔ Submit your assignment

Due Feb 11, 11:59 PM IST

✔ Receive grade

To Pass 80% or higher

👍 Like👎 Dislike📧 Report an issue

✔ Congratulations! You passed!

Grade received 100%

Latest Submission Grade 100%

To pass 80% or higher

Go to next item

The questions below are specifically about the pulp library. You are required to take a look at the ungraded lab that has a tutorial on pulp python library. Also see documentation here: [pulp documentation](https://www.coin-or.org/PuLP/pulp.html) 🔗

1. What does the statement2 / 2 points

```
x_var = pulp.LpVariable('x', lowBound=0, upBound=10, cat='Continuous')
```

achieve? Select all the correct answers. See documentation here: <https://www.coin-or.org/PuLP/pulp.html> 🔗

✔ Setting cat=Continuous is optional since the category is set to continuous by default.

✔ Correct
Correct

✔ It adds a decision variable 'x' to the problem.

✔ Correct
Correct. The first argument is the name.

✔ In a linear programming problem, every decision variable should have cat = Continuous

✔ Correct
Correct: setting cat = Integer or cat = Binary will change it to an integer linear program (to be covered next module).

❑ Setting the optional parameters lowBound=0 has no effect whatsoever.

2. Consider the following code snippet:5 / 5 points

```
1 from pulp import *
2 mdl = LpProblem('my_problem', LpMaximize)
3 x = LpVariable('x', lowBound = 0)
4 y = LpVariable('y', lowBound = 0, upBound = 4)
5 mdl += (x + 2 * y)
6 mdl += x <= y + 3
7 mdl += x >= y - 1
8 mdl += x - 2 * y <= 5
9 status= mdl.solve()
10 print(x.value())
11 print(y.value())
12 print(mdl.objective.value())
```

Select all the correct facts about the code snippet above. Make sure that no incorrect facts are selected.

✔ The linear programming problem seeks to maximize the objective $x + 2 * y$

✔ Correct
Correct

✔ The LP problem as entered in the code snippet is bounded.

✔ Correct
Indeed, both x, y are bounded. Notice that any feasible point must satisfy $y \leq 4, x \leq 7$.

✔ The constraint $x - 2y \leq 5$ is part of the linear program.

✔ Correct
Correct: see line 8

✔ The LP problem as entered in the code snippet is feasible

✔ Correct
Yes, $x = 0, y = 0$ satisfies all constraints.

❑ No objective is set in the code snippet

❑ The linear programming problem seeks to minimize the objective $x + 2 * y$

3. Consider again the code snippet from the previous problem5 / 5 points

```
1 from pulp import *
2 mdl = LpProblem('my_problem', LpMaximize)
3 x = LpVariable('x', lowBound = 0)
4 y = LpVariable('y', lowBound = 0, upBound = 4)
5 mdl += (x + 2 * y)
6 mdl += x <= y + 3
7 mdl += x >= y - 1
8 mdl += x - 2 * y <= 5
9 status= mdl.solve()
10 assert status == LpStatusOptimal
11 print(x.value())
12 print(y.value())
13 print(mdl.objective.value())
```

Select all the correct choices from the list below

✔ The call to y.value() made before the call to mdl.solve() will return None

✔ Correct
Correct

✔ The call to x.value() provides the value of the variable x

✔ Correct
Correct

✔ The assert statement in line 10 assert status == IpStatusOptimal checks if solving the problem has yielded an optimal solution.

✔ Correct
Correct

✔ The statement status=mdl.solve() invokes the solver to solve the problem and status can then be used to tell if the problem was feasible or not.

✔ Correct
Correct

4. Consider the code snippet below:1 / 1 point

```
1 from pulp import *
2 mdl = LpProblem('n_problem', LpMaximize)
3 n = 15
4 vars = [ LpVariable(f'x{i}', lowBound = 0.0) for i in range(n)]
5 mdl += lpSum(vars)
6 for i in range(14):
7     mdl += vars[i] - vars[i+1] <= i
8     mdl += vars[i] - vars[i+1] >= -i
9 status = mdl.solve()
10 if status == LpStatusOptimal:
11     print('Optimal solution found!')
12     print([vi.value() for vi in vars])
13     print(mdl.objective.value())
14 elif status == LpStatusInfeasible:
15     print('Infeasible problem')
16 elif status == LpStatusUnbounded:
17     print('Unbounded problem')
18 else:
19     print('Unknown status')
```

✔ The objective for this problem is the sum of all the variables in the list vars

✔ Correct
Correct: lpSum is defined in the pulp library -- read the documentation here: <https://www.coin-or.org/PuLP/pulp.html#pulp.lpSum> 🔗

❑ The problem is infeasible.

❑ The problem has an optimal solution

✔ Line 4 of the problem declares n=15 variables and stores them in the list vars

✔ Correct
Correct: you should familiarize yourself with list comprehensions -- https://www.w3schools.com/python/python_lists_comprehension.asp 🔗

✔ The problem is unbounded.

✔ Correct
Indeed: there is no upper bound to any of the variables in the list vars

❑ The problem has the constraint vars[10] - vars[11] <= 11

